

In harmony with gpt-oss

Borislav Mavrin

borislav.mavrin@gmail.com

Abstract

No one has independently reproduced OpenAI’s published scores for **gpt-oss-20b** with tools¹, because the original paper discloses neither the tools nor the agent harness. We reverse-engineered the model’s in-distribution tools: when prompted without tool definitions, **gpt-oss** still calls tools from its training distribution with high statistical confidence – a strong prior, not a hallucination. We then built a native harmony agent harness² that encodes messages in the model’s native format, bypassing the lossy Chat Completions conversion. Together, these yield the first independent reproduction of OpenAI’s published scores: 60.4% on SWE Verified HIGH (published 60.7%), 53.3% MEDIUM (53.2%), and 91.7% on AIME25 with tools (90.4%).

Introduction

Many open-source models run on consumer PCs, but only a few can solve coding tasks well: **Devstral Small 2**, **GLM-4.7-Flash**, **Qwen3.5 27b+**, **gpt-oss-20b**.

Practitioners naturally pick the largest model size that "fits" on their GPU. However, the word "fits" deceives. For example, OpenAI claims that "gpt-oss-20b only requires 16GB."^[4] However, even if GPU VRAM is enough to load the model weights, a coding agent requires more. Coding tasks require long contexts of 128k+, which demands more VRAM. Keeping the KV cache across agent turns rather than recomputing it from scratch requires even more VRAM. So 16GB falls short for agentic use. These memory demands further constrain which LLM a practitioner can choose for coding agent tasks.

Latency and throughput also matter. Parameter count alone does not predict them. Architecture matters too – a dense model like **Devstral Small 2** requires more compute than an equally sized MoE model, which activates fewer parameters per forward pass.

Among efficient models that "fit" on a consumer GPU, we then choose by coding capability. SWE-bench is among the most meaningful³ benchmarks because it mirrors real workflows: models must read files, run tests, and edit code through a shell across multiple turns⁴.

Given these factors, **gpt-oss-20b** still offers the best balance of published benchmarks and efficiency among viable local coding models (Table 1), even though it was released in August 2025.

However, published SWE Verified scores may not reproduce under a different agent harness. SWE-bench does not specify an agent harness – the scaffolding that connects a model to tools like **bash**. Each provider builds its own harness, most of them proprietary, and the harness alone can shift scores by tens of percentage points. **Devstral Small 2** scores 68% under its vendor harness but only 56.4% under the open-source harness on the official SWE leaderboard [9]. For **gpt-oss-120b**, the gap is even wider: 62.4% published versus 26% on the leaderboard⁵.

This mismatch matters beyond leaderboard rankings. When a user’s coding tool wraps the same model in a different, unoptimized harness, the model severely underperforms its published numbers – which may explain why some small coding LLMs that look strong on paper fail to gain traction in practice.

gpt-oss compounds the problem further. Its paper uses a proprietary harness and discloses few details beyond a container tool. It also uses a custom chat format—Harmony. OpenAI’s only published evaluation

¹To the best of our knowledge, as of March 2026

²<https://github.com/borislavmavrin/harmonyagent.git>

³There is another pitfall - dataset contamination problem of SWE Verified[6] - which we only acknowledge here, but we leave it outside the scope of the paper.

⁴Which can easily exceed 100 steps.

Model	Published SWE verified score	SWE Leaderboard score ⁵	Model size	Model architecture
Devstral Small 2	68.0%[3]	56.4%	24B	Dense
GLM-4.7-Flash	59.2%[10]	N/A	31B	MoE
Qwen3.5 27b	72.4%[7]	N/A	27B	Dense
gpt-oss-20b	60.7%[5]	26% ⁶	20B	MoE

Table 1: Model size vs SWE Verified scores

framework, **gpt-evals**, tests AIME25 *without* tools, yet the paper reports far higher numbers *with* tools and never specifies what those tools are. However, tools are crucial. On AIME25, **gpt-oss-20b** scores only 72.1% without tools but 90.4% with tools at MEDIUM reasoning – tools make the difference between mediocre and near-perfect performance. And there are no other published results that reproduce OpenAI scores for **gpt-oss** with tools.⁷ To properly use tools, the model requires native Harmony chat format, which is significantly different from standard Chat Completions format.

Tools

Why does this gap exist? The posted logs for **gpt-oss-120b** reveal a major failure mode: the model calls undefined tools. Specifically, it calls **apply_patch** – an undefined tool – 860 times.⁸ This looks like hallucination, but **apply_patch** is a built-in system tool⁹ – the behavior reflects a strong training prior, not a defect. Each time **gpt-oss** edits a file, it calls **apply_patch** – whether or not the tool is defined. So we added the **apply_patch** tool, borrowed from Codex¹⁰, and this failure mode vanished.

Are there any other in-distribution tools? We reverse-engineered the model’s tool priors. The key insight: when prompted without any tool definitions, the model still attempts to call tools from its training distribution. We exploited this to discover tool names and schemas, then cross-referenced text descriptions with actual tool calls to filter confabulations from real parameters.

Pipeline:

1. We extracted 5683 intent–reasoning–tool tuples from raw agent logs.
2. We fed 651 reasoning strings to the model with no tool definitions and recorded which tools it attempted to call; the top prior tools were **container.exec** (4.8%), **repo_browser.print_tree** (3.4%), and **repo_browser.apply_patch** (1.2%).
3. Targeted sampling with specific prompts pinned down argument schemas for each discovered tool (240 samples across four tool groups).
4. We defined the discovered tools and validated them against no-tool baselines.
5. We asked the model directly to list **repo_browser.*** tools and cross-referenced text mentions with actual calls from all prior steps (160 samples).
6. We forced the model to call each tool by name to collect canonical schemas with bootstrap confidence intervals (405 samples, 334 with tool calls).
7. Finally, we asked the model to describe each tool’s schema in natural language and filtered confabulated parameters against actual calls (405 samples).

All confidence intervals reported below are 95% bootstrap CIs (1,000 resamples).

⁵<https://www.swebench.com>

⁶Leaderboard entry is for **gpt-oss-120b**, **gpt-oss-20b** should be no higher.

⁷Despite its name [1] does not use tools in its evaluation.

⁸<https://www.swebench.com/>

⁹<https://github.com/openai/gpt-oss>

¹⁰<https://github.com/openai/codex>

Cross-referencing text mentions with actual calls. The model mentioned 170+ distinct `repo_browser` `.*` tool names in text, most at under 2% – these are confabulations, not in-distribution tools. When we cross-reference text mention rates with actual tool calls from all baseline experiments, the real tools emerge (Table 2).

Tool	Text mentions [95% CI]	Actual calls	Verdict
<code>print_tree</code>	28.1% [21.2, 35.0]	101	confirmed
<code>search</code>	34.4% [27.5, 42.5]	11	confirmed
<code>open_file</code>	44.4% [36.9, 52.5]	3	confirmed
<code>apply_patch</code>	2.5% [0.6, 5.0]	8	confirmed
<code>read_file</code>	25.0% [18.8, 31.9]	1	likely alias
<code>list_files</code>	10.6% [6.2, 15.6]	2	likely alias
<code>delete_file</code>	8.8% [5.0, 13.1]	0	confabulated
<code>write_file</code>	7.5% [3.8, 11.9]	0	confabulated

Table 2: Cross-reference of text-mentioned `repo_browser` tools versus actual tool calls ($n = 160$ text samples, $n = 1201$ baseline samples). Tools with both text mentions and observed calls are confirmed; tools described but never called are confabulations.

Schema robustness. When explicitly asked to call each tool by name, 100% of calls produce valid JSON. Table 3 shows the parameter presence rates for the three primary tools. Alias parameters such as `file_path` (25% in `open_file`), `pattern` (5–7% in `search`), and `start_line/end_line` (8% in `read_file`) appear at under 25% and should be accepted but not required. The model confabulates 3–10 plausible parameters per tool (e.g., `encoding`, `case_sensitive`, `recursive`) that it describes in text but never uses in actual calls.

Parameter	<code>print_tree</code> ($n = 108$)	<code>search</code> ($n = 38$)	<code>open_file</code> ($n = 32$)
<code>path</code>	100% [100, 100]	97% [92, 100]	72% [56, 88]
<code>depth</code>	99% [97, 100]	—	—
<code>query</code>	—	95% [87, 100]	—
<code>max_results</code>	—	53% [37, 68]	—
<code>line_start</code>	—	—	78% [62, 91]
<code>line_end</code>	—	—	72% [56, 88]

Table 3: Schema robustness: parameter presence rates when the model is asked to call each tool by name (Step 5, 95% bootstrap CIs).

Discovered tool inventory. The model’s in-distribution `repo_browser` namespace contains three functional groups plus `apply_patch` (Table 4). Plus `container.exec` for shell execution.

Alias tools such as `list_dir`, `list_directory`, `read_file`, and `find` share schemas with their primary counterparts and collapse into the three core tools when tools are properly defined.

Group	Primary tool	Aliases	Canonical schema
Directory listing	<code>print_tree</code>	<code>list_files</code> , <code>list_dir</code>	<code>{path, depth}</code>
Content search	<code>search</code>	<code>find</code>	<code>{path, query, max_results?}</code>
File reading	<code>open_file</code>	<code>read_file</code>	<code>{path, line_start?, line_end?}</code>
Patch	<code>apply_patch</code>	—	<code>{patch}</code>

Table 4: Discovered in-distribution `repo_browser` tools with canonical schemas, plus `container.exec` as a flat tool.

Validation: baseline versus with tools. Defining the discovered tools in the system message dramatically increases tool call rates (Table 5). When we defined these tools in the developer message – as OpenAI’s cookbook documents[2] – the model still failed to call them reliably. But when we moved them to the system message, the model called them reliably.

Tool	Prompt type	Baseline	With tools	Lift
<code>print_tree</code>	explore	29.4% [20, 40]	98.6% [96, 100]	3.4×
<code>search</code>	search	3.8% [0, 9]	58.8% [48, 69]	15×
<code>open_file</code>	read code	1.2% [0, 4]	21.2% [13, 30]	18×

Table 5: Tool call rates without tools defined (baseline) versus with tools in the system message (95% bootstrap CIs).

Behavioral observations. `print_tree` dominates as the default first action even for search or file-reading prompts, matching the agent log pattern where directory listing is always the first command. `open_file` has an overall call rate of only 21% but reaches 60% when the prompt contains both a file path and line numbers. `search` is reliably triggered by prompts with search-intent keywords such as “search”, “grep”, or “find definition”. When the three primary tools are formally defined, alias variants (`list_files`, `read_file`, `find`) vanish – the model uses the defined tools exclusively.

Harmony agent

A second major factor that explains the gap between published and reproduced scores: OpenAI trained `gpt-oss-20b` on the harmony format, which differs radically from the standard OpenAI Chat Completions format. Mapping one format to the other is complex. Yet no existing backend provides a native harmony API (to the best of our knowledge); they all convert to the OpenAI Chat Completions API or OpenAI Response API format, which may distort model output. Worse, none can define tools in harmony format.

We solved this by implementing the Harmony Agent. We built on the simple yet powerful `mini-swe-agent`.^[8]¹¹

Our harness encodes and decodes messages in harmony’s native token format through the `openai_harmony` library, so it never converts to Chat Completions format. The Harmony agent consumes the raw tokenized output of `gpt-oss`, i.e. `<|start|>header<|message|>content<|end|>`. It starts each run with three messages: a system message sets the model identity, reasoning effort, and tools in TypeScript-like syntax; a developer message defines the task instructions; a user message states the task. The agent then loops: render the conversation to tokens, query the model, parse the response, and either execute a tool call or terminate.

Special tokens (`<|start|>`, `<|message|>`, `<|end|>`) delimit each message; every message carries a role and a channel. The model responds on three channels: `analysis` for chain-of-thought reasoning, `commentary` for tool calls, and `final` for the user-facing answer. `<final>` message terminates the task. Because channels separate message types, the harness routes them accordingly.

In harmony, the system or developer message defines tools once, saving many tokens; the OpenAI Chat Completions format repeats them every turn. Figure 2 shows token usage across all Harmony trajectories for 500 SWE Verified instances with `gpt-oss-20b` at Medium and High reasoning. For Chat Completions estimates, we simply add the number of tokens for tool definitions to each turn. This difference is especially pronounced for High reasoning.

When the model calls a tool, the harness reads the tool name from the recipient field, parses the JSON arguments, executes the tool in a sandboxed container, and appends the result as a tool-role message. The harness validates each step and raises corresponding custom exceptions. Exceptions fall into two categories (following the SWE-bench paper): Terminating and NonTerminating. Terminating exceptions halt the run because the harness cannot recover by retrying. The harness retries malformed calls and unparseable arguments up to ten times, but terminates the run when the context window overflows or the agent exceeds its step limit. NonTerminating exceptions signal errors the model can likely recover from on retry.

¹¹<https://github.com/SWE-agent/mini-swe-agent>



Figure 1: Harmony agent.

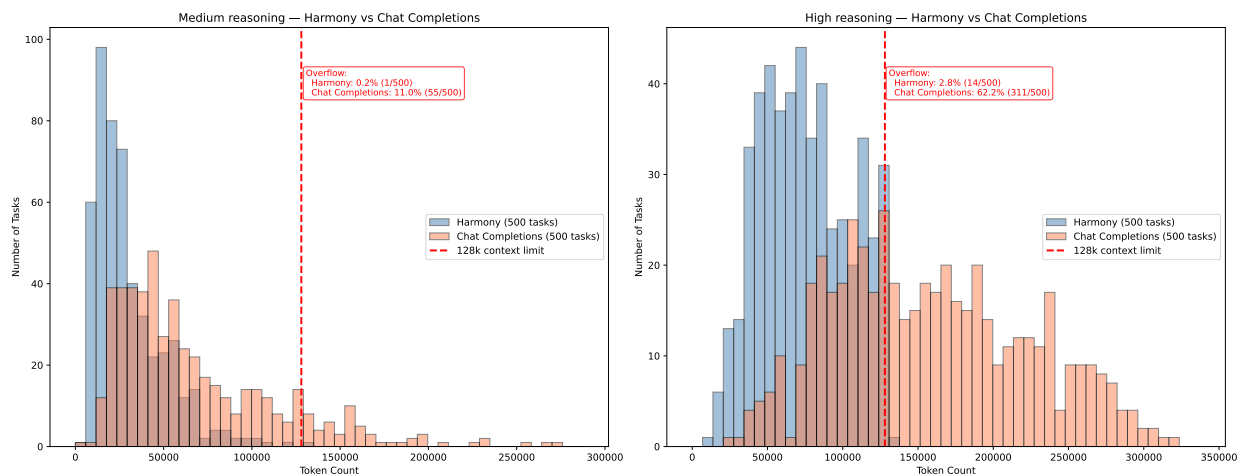


Figure 2: Harmony vs chat completions token usage.

NonTerminatingException:

- **LongGeneration**: Raised when generation exceeded `max_tokens`.
- **HarmonyParsingError**: Harmony error when parsing LLM completion.
- **HarmonyMessageMissingChannel**: Raised when a non-system harmony message is missing a channel.
- **MultipleReasoningMessages**: Raised when multiple reasoning messages.
- **MultipleFinalMessages**: Raised when multiple final messages are found.
- **MultipleToolCalls**: Multiple tool calls.
- **NoToolCallNoFinalMessage**: Raised when no tool call and no final message are received.
- **ToolCallAndFinalMessage**: Raised when both tool call and final message are received.
- **ToolNameParsingError**: Raised when tool name failed to parse.
- **UnknownToolCalled**: Unknown tool called.
- **UnknownToolCallArg**: Tool call has unknown argument.
- **ToolCallArgParsingError**: Failed to parse tool call arguments.
- **ExecutionTimeoutError**: Raised when the action execution timed out.

TerminatingException:

- **Submitted**: Raised when the LM declares that the agent has finished its task.
- **LimitsExceeded**: Raised when the agent has reached its cost or step limit.
- **MaxContextWindowOverflow**: Raised when max context window of the model is reached.
- **UnexpectedFinishReason**: Raised when OpenAI completions finishes with unknown reason.
- **MaxNewTokensExceeded**: Raised when `max_tokens` is exceeded.
- **RetrialsExceeded**: Raised when retrials on `NonTerminatingExceptions` are exceeded.

This exception system is crucial: OpenAI recommends running `gpt-oss` at `temperature 1` and `top_p 1`¹², so the model may deviate from the harmony format. If we apply the harmony format and use in-distribution tools, the model rarely deviates, and `gpt-oss` recovers on retry.

Evaluation

The real test of our approach is how well we can reproduce the published results. We ran two benchmarks – AIME25 and SWE Verified – and achieved results that match those in OpenAI’s paper on `gpt-oss-20b`. All experiments were run with `vLLM 0.14.1` inference with: `-tensor-parallel-size 1 -max-model-len 131072`. All results are pass@1. In the HIGH reasoning setting for SWE Verified, we only retried `MaxContextWindowOverflow` exceptions until the agent reached a final message, since verbose reasoning traces often overflow the 128k context window. We could alternatively compact the context, but that approach is more complex and we left it for future work.

¹²<https://github.com/openai/gpt-oss>

Benchmark	Published score[5]	HarmonyAgent score	95% Bootstrap CI
SWE Verified HIGH reasoning	60.7%	60.4%	[56.2%, 64.8%]
SWE Verified MEDIUM reasoning	53.2%	53.3%	[49.3%, 57.7%]
AIME 2025 MEDIUM reasoning with tools	90.4%	91.7%	[87.5%, 95.0%]

Table 6: OpenAI published results vs HarmonyAgent results (with 95% bootstrap CIs, 1,000 resamples).

Conclusions

The gap between published and reproduced scores for **gpt-oss-20b** is not a model deficiency – it is a harness deficiency. We closed that gap with two changes: we provided the model’s in-distribution tools (**container.exec**, **repo_browser** and **apply_patch**), and we implemented a native Harmony harness rather than relying on Chat Completions format. Our scores match OpenAI’s published numbers within confidence intervals on both SWE Verified and AIME25.

This result generalizes. Any model trained on a specific tool set and message format might underperform when a generic harness omits those tools or converts that format. This is especially true for **gpt-oss**: the 36-point gap between **gpt-oss-120b**’s published score and its SWE leaderboard entry is the expected cost of harness mismatch. When practitioners deploy open-source models in coding agents and see disappointing results, the model may not be at fault – the harness may be.

Without an open harness, practitioners dismissed **gpt-oss-20b** as weaker than it is. Open-source models deserve correct open-source evaluation harnesses.

References

- [1] Ziqian Bi, Keyu Chen, Chiung-Yi Tseng, Danyang Zhang, Tianyang Wang, Hongying Luo, Lu Chen, Junming Huang, Jibin Guan, Junfeng Hao, Xinyuan Song, and Junhao Song. Is GPT-OSS good? a comprehensive evaluation of OpenAI’s latest open source models. *arXiv preprint arXiv:2508.12461*, 2025.
- [2] Dominik Kundel. OpenAI harmony response format, 2025. URL <https://developers.openai.com/cookbook/articles/openai-harmony>. OpenAI Developer Cookbook.
- [3] Mistral AI. Devstral-Small-2-24B-Instruct-2512 model card, 2025. URL <https://huggingface.co/mistralai/Devstral-Small-2-24B-Instruct-2512>. Hugging Face.
- [4] OpenAI. Introducing gpt-oss, August 2025. URL <https://openai.com/index/introducing-gpt-oss/>. OpenAI.
- [5] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- [6] OpenAI. Why swe-bench verified no longer measures frontier coding capabilities, February 2026. URL <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>. OpenAI.
- [7] Qwen. Qwen3.5 model card, February 2026. URL <https://huggingface.co/Qwen/Qwen3.5-27B>. Hugging Face.
- [8] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [9] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. mini-swe-agent. <https://github.com/SWE-agent/mini-swe-agent>, 2026. Version number, accessed on Day Month Year.

- [10] Z.AI. GLM-4.7-Flash model card, 2025. URL <https://huggingface.co/zai-org/GLM-4.7-Flash>. Hugging Face.